Capability-Based WebAssembly Runtime

Group 10
Rui Xavier **Jendrik Engels** **Darius Mureșan**

May 27, 2026

Abstract

Executing untrusted C code safely is a critical challenge in modern systems, as traditional coarse-grained process isolation fails to prevent memory safety exploits from escalating via ambient authority. In this paper, we present a secure, capability-based WebAssembly runtime designed to enforce three foundational Language-Based Security properties: Software-Based Fault Isolation, Object Capabilities, and Execution Monitoring. By compiling adversarial C payloads to the `wasm32-wasip1` target and embedding them within a zero-ambient-authority Python host, we demonstrate how WebAssembly's split-memory architecture inherently neutralizes buffer overflows and indirect call hijacking. Furthermore, we address the Confused Deputy problem by extending the WebAssembly System Interface to enforce fine-grained, per-file capabilities through ephemeral sandboxing. Finally, we implement fuel-based liveness policies and a host-mediated communication channel to safely execute and compose parallel untrusted modules. Our evaluation confirms that this architecture effectively contains memory corruption and restricts authority without requiring complex assembly rewriting or native runtime instrumentation.

Contents

1 Introduction **3**

2 Background and Threat Model **3**
2.1 WebAssembly and SFI . . . 3
2.2 The WebAssembly System Interface (WASI) . . . 3
2.3 The Confused Deputy and Ambient Authority . . . 3
2.4 Threat Model and TCB . . . 4
2.4.1 Trust Boundaries and the TCB . . . 4
2.4.2 Attacker Capabilities and Objectives . . . 4
2.4.3 Enforced Security Policies . . . 4

3 Design and Implementation **4**
3.1 Enforcing SFI and Control Flow Integrity . . . 4
3.2 Execution Monitoring and Liveness . . . 5
3.3 Object Capabilities and WASI Mediation . . . 5
3.3.1 Zero Ambient Authority via WASI . . . 5
3.3.2 Ephemeral Sandboxing for Fine-Grained Access Control . . . 5
3.3.3 Network Capability Restriction . . . 6
3.4 Compositional Security and Host-Mediated Channels . . . 6
3.4.1 Filesystem Bridge with Capability Rotation . . . 6
3.4.2 Host-Mediated Channel for Parallel Sandboxes . . . 6
3.4.3 Why Direct Memory Pointers Violate SFI . . . 6

4 Evaluation and Security Testing **7**
4.1 Evaluating Memory Integrity and CFI . . . 7
4.1.1 Test 02: Mitigating Stack-Based Buffer Overflows . . . 7
4.1.2 Test 04: Preventing Function Pointer Hijacking . . . 7
4.2 Evaluating Liveness Guarantees . . . 7
4.2.1 Test 05: Defeating CPU-Exhaustion Denial of Service . . . 7
4.3 Evaluating Capability Boundaries . . . 7
4.3.1 Test 03: Preventing Filesystem Escape via Capabilities . . . 7

4.3.2 Test 06: Blocking Network Access 8
4.3.3 Tests 07 & 08: Fine-Grained Per-File Capabilities 8
4.4 Evaluating Sandbox Composition 8
4.4.1 Test 09: Filesystem Bridge 9
4.4.2 Test 10: Memory Isolation Between Parallel Sandboxes 9

5 Related Work **9**

6 Conclusion **10**

1 Introduction

The execution of untrusted C code presents a fundamental security challenge. Traditional native binaries execute with the ambient authority of the host process, relying solely on coarse-grained operating system boundaries for isolation. Within this model, memory safety violations such as buffer overflows can easily escalate into arbitrary code execution, compromising the host environment. To mitigate these risks, this project implements a secure, capability-based runtime for untrusted C code utilizing WebAssembly (Wasm) and a custom Python host leveraging the Wasmtime library.

Rather than attempting to secure native x86 execution through complex assembly rewriting, we compile deliberately vulnerable C programs to the `wasm32-wasip1` target. This architecture serves as our untrusted guest environment, allowing us to enforce and demonstrate three core Language-Based Security (LBS) properties:

- **Software-Based Fault Isolation (SFI):** By confining execution to Wasm's linear memory and split-stack architecture, we structurally neutralize classical memory exploits. We demonstrate how stack buffer overflows and indirect call hijacking, which would succeed natively, are successfully contained without host compromise.
- **Object Capabilities:** We directly address the Confused Deputy problem by stripping the execution environment of ambient authority. By mediating the WASI, we implement a strict capability model where access to any resource must be explicitly delegated by the host.
- **Execution Monitoring:** We enforce liveness and prevent Denial of Service (DoS) attacks by metering computational resources. Using a fuel-based instruction counter, we demonstrate an enforceable security policy that guarantees guest termination.

By maintaining a strict physical boundary between the Python host (our Trusted Computing Base (TCB)) and the adversarial C payloads, this paper demonstrates that a tightly controlled, zero-ambient-authority runtime can effectively neutralize fatal application-level vulnerabilities.

The complete source code for our runtime, including the trusted host implementation and the adversarial C payloads used for evaluation, is open-source and publicly available [Xavier et al., 2026].

2 Background and Threat Model

2.1 WebAssembly and SFI

Wasm is a portable, low-level bytecode format originally designed for secure execution within web browsers, but increasingly utilized as a generic sandboxing environment. A core feature of Wasm is its default isolation: a compiled module operates entirely within a self-contained virtual machine and has no implicit access to the host operating system's memory or execution context.

This architecture naturally enforces SFI, a concept introduced in the foundational work by [Wahbe et al., 1993]. SFI proposes isolating untrusted software modules to prevent memory faults from escalating. Traditional C programs compiled to native architectures (such as x86) share a unified address space for the stack, heap, and control flow structures (e.g., return addresses). This layout inherently allows memory corruption vulnerabilities, such as stack-based buffer overflows, to hijack control flow.

Conversely, Wasm enforces SFI structurally through a split-memory design. The heap and local variables are confined to a contiguous, bounds-checked "linear memory" space. Crucially, this linear memory is completely decoupled from the virtual machine's internal execution stack, where return addresses and control-flow data are stored. Consequently, while a buffer overflow can corrupt adjacent data within the linear memory, it is structurally impossible for an attacker to overwrite saved return addresses to achieve arbitrary code execution.

2.2 The WebAssembly System Interface (WASI)

Because a Wasm module is strictly isolated, it cannot perform useful computations without a mechanism to interact with the outside world (e.g., file I/O, networking). The WASI was developed to bridge this gap, serving as an Application Binary Interface (ABI) that provides POSIX-like system calls to the guest module.

However, unlike traditional POSIX environments that rely on ambient authority (privileges inherently granted to a process based on the executing user), WASI is fundamentally designed around capability-based security. By default, a Wasm module initialized with WASI has zero ambient authority, it cannot open a file or a network socket unless the host explicitly passes a capability (a file descriptor token) for that specific resource into the sandbox.

2.3 The Confused Deputy and Ambient Authority

As identified by [Hardy, 1988], systems relying on ambient authority are inherently vulnerable to the Confused Deputy problem, where a privileged program is manipulated by an unprivileged entity into misusing its authority

(e.g., path traversal attacks).

Object Capability models solve this by replacing ambient authority with unforgeable tokens of authority. Our runtime leverages WASI's capability model to ensure the guest environment possesses no implicit access to the host filesystem. If an untrusted C program attempts to access a file or network resource it was not explicitly granted a capability for, the runtime acts as a reference monitor and blocks the execution.

2.4 Threat Model and TCB

To rigorously evaluate our Language-Based Security guarantees, we define a strict threat model that separates the TCB from the adversarial execution environment. Within this model, our runtime acts as an unbypassable Reference Monitor enforcing formal security policies against a powerful adversary.

2.4.1 Trust Boundaries and the TCB

The physical and logical boundary between the host and the guest defines our system's security perimeter. Our TCB consists of the Python host environment, the Wasmtime virtual machine, and the underlying host operating system. We assume the TCB is implemented correctly, is free of vulnerabilities, and is unconditionally trusted. The TCB operates as a Reference Monitor: it initializes the sandboxes, delegates capabilities, and mediates all cross-domain operations.

2.4.2 Attacker Capabilities and Objectives

We assume the untrusted guest module is fully adversarial. Our threat model grants the attacker the following capabilities:

- **Total Memory Control:** The attacker has arbitrary read and write access to all data within the Wasm linear memory, simulating successful exploitation of severe memory corruption vulnerabilities (e.g., buffer overflows) at the C level.
- **Execution Control:** The attacker controls the logic of the compiled Wasm binary and can attempt to manipulate execution state (e.g., corrupting function pointers) to hijack control flow.
- **Environmental Knowledge:** The attacker possesses full knowledge of the runtime configuration, capability boundaries, and isolation mechanisms.

Given these capabilities, the adversary's objectives (the threats) are threefold: (1) to escape the memory sandbox and achieve arbitrary code execution on the host; (2) to manipulate the host into acting as a Confused Deputy to access unauthorized files or networks; and (3) to perform a CPU-exhaustion Denial of Service (DoS) attack to render the TCB unresponsive.

2.4.3 Enforced Security Policies

To neutralize these threats, our Reference Monitor enforces a strict set of Language-Based Security policies over the guest's execution trace. In LBS, policies are traditionally categorized as either *safety properties* ("nothing bad ever happens") or *liveness properties* ("something good eventually happens"). Our runtime mathematically guarantees the following:

- **Memory Safety Policy (SFI):** A *safety property* mitigating Threat 1. It guarantees that the guest execution trace cannot name, address, or manipulate memory outside its contiguous linear memory bounds, structurally protecting the VM's internal control-flow stack from compromise.
- **Capability-Based Access Policy:** A *safety property* mitigating Threat 2. It enforces zero ambient authority: the guest cannot interact with any external file or network resource unless it explicitly invokes an unforgeable capability token (a file descriptor) that was deliberately delegated by the TCB.
- **Execution Liveness Policy:** A *liveness property* mitigating Threat 3, dictating that untrusted guest execution must eventually terminate. Because unconditional termination is undecidable (the Halting Problem), we translate this into an enforceable safety property via deterministic fuel bounding: the guest is structurally forced to halt or trap within a strictly allocated budget of computational instructions.

3 Design and Implementation

Our runtime architecture is designed to intercept and neutralize vulnerabilities before they can violate the foundational properties of the TCB. This section details the mechanisms used to enforce SFI, Execution Monitoring, and Object Capabilities.

3.1 Enforcing SFI and Control Flow Integrity

To demonstrate the structural security of our Wasm runtime, we compiled deliberately vulnerable C programs containing classical memory corruption bugs to the `wasm32-wasip1` target.

On a native x86 architecture, a stack-based buffer overflow via an unsafe function like `strcpy` typically allows an attacker to write past the bounds of a local variable and overwrite the saved return pointer on the execution

stack, thereby hijacking control flow. We mitigate this by leveraging Wasm's split-stack architecture. Wasm segregates the addressable linear memory (where the C heap and stack-allocated data buffers reside) from the VM's internal, unreachable execution stack (where return addresses and control-flow state are managed). Consequently, any C-level pointer arithmetic or unbounded write is inherently restricted to the linear memory bounds.

Furthermore, we enforce coarse-grained Control Flow Integrity (CFI) natively through Wasm's handling of indirect function calls. In native C, function pointers are stored as raw memory addresses. In our Wasm implementation, function pointers are compiled as integer indices into a VM-managed structure called the Table. This Table is completely opaque to the guest's linear memory. When an indirect call is executed via the `call_indirect` instruction, the Wasm runtime acts as a reference monitor, performing a strict type-signature validation. It compares the statically expected signature at the call site against the dynamic signature of the function located at the target index.

3.2 Execution Monitoring and Liveness

Beyond spatial memory safety, a secure runtime must guarantee liveness properties to prevent Denial-of-Service (DoS) attacks. An adversarial guest could execute an infinite loop, monopolizing the host's CPU thread and rendering the TCB unresponsive.

To formalize an enforceable liveness policy, we implemented an execution monitor utilizing the Wasmtime engine's deterministic fuel mechanism. Before invoking an untrusted module, the Python host configures the VM instance with `consume_fuel = True` and injects a strict computational budget (e.g., 50,000 fuel units, where one unit roughly correlates to one executed Wasm instruction). The VM synchronously decrements this counter during execution. If the budget is exhausted, the VM structurally prevents further execution and raises an exception back to the host, effectively bounding the halting problem for untrusted modules.

3.3 Object Capabilities and WASI Mediation

A secure runtime must not only confine memory accesses (SFI) and bound computation (Execution Monitoring), but also restrict which host resources an untrusted guest may interact with. Our design enforces the Object Capability model by ensuring that the guest execution environment possesses zero ambient authority: no file, directory, or network resource is accessible unless the Python TCB has explicitly delegated a capability for it.

3.3.1 Zero Ambient Authority via WASI

By default, a freshly instantiated Wasm module linked against the WASI has no access to the host filesystem, network stack, or environment variables. Every resource interaction must be mediated through file descriptor capabilities that the host passes into the sandbox at initialization. Our Python host exploits this property by constructing a `WasiConfig` with no pre-opened directories and no inherited handles. The guest therefore begins execution in a state of complete isolation, from its perspective, the outside world does not exist.

To grant access to a specific resource, the host calls `wasi.preopen_dir(host_path, guest_path)`, which maps a host-side directory into the guest's virtual namespace. Crucially, the guest can only name paths relative to this pre-opened root; it cannot forge or escalate capabilities beyond what was explicitly delegated. Path traversal strings such as `../secret.txt` are resolved relative to the pre-opened root and cannot escape the capability boundary, because WASI's path resolution is confined to the set of granted file descriptors.

3.3.2 Ephemeral Sandboxing for Fine-Grained Access Control

A limitation of WASI P1's native capability model is that it operates at directory granularity: pre-opening a directory grants the guest access to every file within it. This coarse-grained delegation violates the Principle of Least Privilege[Saltzer and Schroeder, 1975] when a guest should only access a specific subset of files, or when different operations (read vs. write) should be permitted on different files.

To address this, we implement an *ephemeral sandbox* mechanism. Rather than pre-opening the real `sandbox_data/` directory, the TCB performs the following steps before each guest invocation:

1. **Policy Lookup:** The host consults a declarative `SECURITY_POLICY` dictionary that maps each Wasm binary to a set of per-file, per-operation capabilities. For example, `prog1.wasm` is granted read-only access to `x.txt`, while `prog2.wasm` receives read-write access to `y.txt`.
2. **Ephemeral Directory Construction:** Using Python's `tempfile.TemporaryDirectory()`, the TCB creates a fresh, empty directory that exists only for the duration of the guest's execution.
3. **Selective File Copying:** Only the files listed in the guest's policy are copied (`shutil.copy2`) into the ephemeral directory. Files not mentioned in the policy are never present in the sandbox and are therefore invisible to the guest, they cannot even be named.
4. **Per-Operation Enforcement via OS Permissions:** For each copied file, the TCB applies native OS-level permissions using `os.chmod`. A read-only capability (`"r"`) translates to mode `0o444`, while a read-write capability (`"rw"`) translates to `0o644`. Because the Wasmtime engine respects the underlying OS

file permissions when executing WASI system calls, any attempt by the guest to perform an unauthorized operation (e.g., writing to a read-only file) will fail with a permission error at the OS level.

5. **Capability Delegation:** The ephemeral directory is pre-opened as `/data` in the guest's WASI namespace. The guest believes this curated view is the entire filesystem available to it.
6. **Cleanup:** When the guest terminates (normally or via a trap), the `with` block's context manager automatically destroys the ephemeral directory and all its contents, leaving no residual state.

This design composes naturally with WASI's existing capability model: we do not modify or extend the WASI specification itself, but instead use the host's authority over the physical filesystem to construct a minimal view that enforces the desired policy before the guest ever executes.

3.3.3 Network Capability Restriction

Network access is restricted through two complementary mechanisms. First, at the *structural* level, WASI P1 does not expose any socket primitives whatsoever. Functions such as `socket()`, `connect()`, and `bind()` are absent from the WASI sysroot, which means that a C program compiled to `wasm32-wasip1` cannot even compile code that references these functions. Network access is therefore impossible at the language level, not merely blocked at runtime.

Second, at the *capability* level, our host does not pre-open any system directories such as `/dev`, `/proc`, or `/etc`. A guest that attempts to access pseudo-filesystem network interfaces (e.g., `/dev/tcp`, `/proc/net/tcp`) or network configuration files (e.g., `/etc/hosts`) will find that these paths simply do not exist within its namespace. The combination of absent API surface and absent capabilities makes network-based exfiltration or lateral movement structurally infeasible.

3.4 Compositional Security and Host-Mediated Channels

The properties established so far concern a single untrusted module executing in isolation. However, a practical runtime must support composition. This means multiple mutually distrusting modules cooperating to perform useful work. This introduces a new adversarial surface, as each module must be treated as hostile not only towards the host but also towards the other modules. Our goal is to permit two sandboxes to exchange data while guaranteeing that neither can read the other's private state, corrupt it, or escalate its own authority. We implement two different mechanisms: a filesystem-mediated bridge with rotating capabilities, and a host-mediated communication channel for sandboxes executing in parallel.

3.4.1 Filesystem Bridge with Capability Rotation

The first mechanism reuses the object-capability approach of section 3.3 to arrange an asynchronous Producer–Consumer handoff. Rather than granting both modules simultaneous read/write access to a shared location, which would permit a race in which the Consumer overwrites the Producer, the host treats access to the shared region as a capability that is granted for exactly one phase of the workflow and then revoked. The host first grants the Producer write authority over the shared file, executes it to completion, and then dynamically rotates the permission (via `os.chmod`) to grant the Consumer read-only access before executing it. Because capability tokens in our model are unforgeable and cannot be escalated from within the guest, the Consumer is structurally incapable of writing back into the bridge once the Producer's phase has concluded, and neither module ever holds authority beyond the minimum required for its role. Information flows in a single direction, that is controlled by the host.

3.4.2 Host-Mediated Channel for Parallel Sandboxes

The second mechanism addresses the more demanding case of two sandboxes executing oncurrently, each requiring a live communication mechanism. Each guest is instantiated in its own Wasmtime `Store`, whci is the per-instance container holding all mutable VM state and includes the guest's private linear memory and is driven on a dedicated operating-system thread. A single Wasmtime `Engine` is shared across both threads. This is sound because the `Engine` is effectively immutable once configured, whereas all mutable per-guest state is restricted to its respective `Store`. Because Wasmtime releases the Python Global Interpreter Lock while executing native guest code, the two modules achieve genuine parallel execution.

Crucially, the two sandboxes do not share any Wasm-level memory. The communication mechanism (called *channel*) is a fixed-size byte buffer owned exclusively by the Python TCB. Each guest is granted access to this buffer. This works due to two imported host functions instead of through a memory import:
`channel_write(offset, value)` and `channel_read(offset)`. Every cross-sandbox byte transfer therefore traps out of the guest VM and into trusted host code, where the host validates the requested `offset` against the channel bounds and acquires a lock to guarantee atomicity before touching the buffer. An out-of-bounds request is rejected and logged. In effect, the host acts as a complete reference monitor on every individual byte that crosses the boundary between the two modules.

3.4.3 Why Direct Memory Pointers Violate SFI

An alternative would expose one sandbox's linear memory directly to the other, for instance by importing the Producer's memory into the Consumer or by handing the Consumer a raw pointer into the Producer's address

space. We reject this design, as it fundamentally violates the SFI guarantee established in section 3.1.

The isolation property rests on a single invariant: a module must be unable to *name* any address outside its own contained region. A pointer into a peer's memory destroys this invariant directly, because it provides the holder with a handle into memory it does not own. Once such a handle exists, the C-level pointer arithmetic would allow the Consumer to traverse the entirety of the Producer's heap, stack-allocated data, and global state. The isolation guarantee would degrade from a structural impossibility to a fragile dependence on the Consumer never computing a hostile offset.

The host-mediated channel preserves SFI because it never connects the two address spaces. The Producer's private state remains within the Producer's linear memory and is unaddressable by the Consumer. Even adversarial pointer manipulation within the Consumer's own linear memory cannot reach the Producer, as no path between the two regions exists at the Wasm level. Composition is therefore achieved without weakening the per-module isolation that the runtime is built to guarantee.

4 Evaluation and Security Testing

We evaluate our runtime against a suite of adversarial C payloads designed to breach memory integrity, capability boundaries, and system liveness. The host script executes these binaries and logs the intercepted violations, confirming our theoretical LBS guarantees in practice.

All ten adversarial test cases, along with the automated execution scripts required to reproduce this evaluation, can be found in our project repository [Xavier et al., 2026].

4.1 Evaluating Memory Integrity and CFI

4.1.1 Test 02: Mitigating Stack-Based Buffer Overflows

To validate SFI against linear memory overflows, the guest executes a payload (`02_sfi_buffer_overflow`) that performs an unbounded `strcpy` operation into a fixed-size local stack buffer, attempting to overwrite a saved return address. The evaluation confirms a critical distinction between a memory corruption occurring and an exploit succeeding.

The out-of-bounds write successfully occurs and corrupts adjacent local variables residing within the Wasm linear memory, confirming that the data stack is contiguous and the write is not bounds-checked at the C level. However, because the malicious payload cannot generate an address that points to the VM's internal execution stack, the return address remains perfectly intact. The program ultimately terminates normally without the attacker gaining control of the instruction pointer, structurally guaranteeing CFI against stack smashing.

4.1.2 Test 04: Preventing Function Pointer Hijacking

To evaluate our CFI guarantees against indirect call hijacking (`04_sfi_indirect`), the guest utilizes a payload that overflows a struct to overwrite an adjacent function pointer. The attacker's objective is to redirect execution to an unauthorized `admin_shell(int)` function that was never intended to be accessible via that pointer.

Upon execution, the attacker successfully corrupts the integer index representing the function pointer. However, when the altered indirect call is invoked, the evaluation demonstrates that the Wasmtime VM successfully intercepts the transition. Because the `admin_shell(int)` function requires an integer argument, its signature does not perfectly match the expected `void()` signature at the call site. The VM aborts the operation and traps with a `RuntimeError: call indirect type mismatch`. The jump is never taken, neutralizing the exploit.

4.2 Evaluating Liveness Guarantees

4.2.1 Test 05: Defeating CPU-Exhaustion Denial of Service

To test the efficacy of our execution monitor (`05_execution_monitoring`), we deployed a malicious payload consisting of a deliberate, infinite `while(1)` loop. Without intervention, this payload would stall the host thread indefinitely.

The Python TCB provisions the sandbox with exactly 50,000 fuel units prior to instantiation. The evaluation shows that the guest spins freely until the instruction counter reaches zero. At that exact moment, the VM forcefully raises an out-of-fuel trap. The host intercepts this trap using a standard `try/except` block, safely terminates the guest instance, and logs the violation without crashing. This confirms that the host is impervious to computationally exhaustive DoS attacks and retains absolute control over its execution thread.

4.3 Evaluating Capability Boundaries

4.3.1 Test 03: Preventing Filesystem Escape via Capabilities

To validate our object capability model against unauthorized filesystem access, the guest (`03_cap_file_access`) acts as a Confused Deputy. It attempts three file operations: (1) reading `/data/public.txt`, which resides inside the pre-opened capability directory; (2) reading `/etc/passwd`, an absolute host path entirely outside the

sandbox; and (3) performing a path traversal via `/data/../secret.txt` to escape the sandbox boundary and reach a file deliberately placed outside `sandbox_data/`.

The evaluation confirms that only the first operation succeeds. The guest reads `public.txt` and prints its contents, demonstrating that legitimately delegated capabilities function correctly. Both unauthorized attempts fail silently, the guest receives a null file handle and reports the denial. Crucially, the path traversal attack is neutralized not by string filtering or path canonicalization, but by the fundamental structure of the capability model: the `../` sequence is resolved relative to the pre-opened root, and because no capability exists for any path outside `/data`, the traversal simply has nowhere to go. The `secret.txt` file is not merely inaccessible, from the guest's perspective, it does not exist.

4.3.2 Test 06: Blocking Network Access

To evaluate network-level isolation, the guest (`06_cap_network_access`) attempts to access three network-related system resources: `/dev/tcp` (a pseudo-device for TCP connections), `/proc/net/tcp` (the kernel's TCP connection table), and `/etc/hosts` (the hostname resolution configuration file).

All three attempts fail with "No such file or directory." The guest terminates normally after reporting each failure. This result confirms that our capability model provides defense in depth against network access. At the API level, WASI P1 does not expose socket primitives, `socket()`, `connect()`, and `bind()` are absent from the sysroot and cannot be compiled against. At the capability level, our host does not pre-open `/dev`, `/proc`, or `/etc`, so even pseudo-filesystem approaches to network interaction are blocked. The guest has neither the system call interface nor the capability tokens required to perform any form of network communication.

4.3.3 Tests 07 & 08: Fine-Grained Per-File Capabilities

To validate our ephemeral sandboxing mechanism beyond coarse-grained directory delegation, we deploy two cooperating test programs with distinct, fine-grained security policies. The policy for `prog1.wasm` grants read-only access to `x.txt`, while the policy for `prog2.wasm` grants read-write access to `y.txt`. Neither program is granted any capability for the other's file.

Test 07 (prog1): The guest successfully reads `x.txt`, confirming that the read capability was correctly delegated. It then attempts to write to `x.txt` by opening it in append mode, which fails with a permission error, the ephemeral copy was created with mode `0o444` (read-only), so the OS rejects the write at the syscall level. Finally, it attempts to read `y.txt`, which fails because `y.txt` was never copied into `prog1`'s ephemeral sandbox and is therefore invisible to the guest.

Test 08 (prog2): The guest successfully writes to `y.txt` and then reads back its contents, confirming that the read-write capability functions correctly. It then attempts to read `x.txt`, which fails because `x.txt` was never placed in `prog2`'s ephemeral directory.

These tests demonstrate three properties of our fine-grained capability system: (1) per-file isolation, each guest sees only the files explicitly listed in its policy; (2) per-operation enforcement, the TCB can grant read access without write access to the same file; and (3) ephemeral confinement, because each guest receives a fresh, minimal copy of the filesystem, there is no possibility of cross-contamination between successive or concurrent invocations.

4.4 Evaluating Sandbox Composition

To validate that our runtime composes mutually distrusting modules without weakening per-module isolation, we evaluate the two communication mechanisms introduced in section 3.4.

We are using a separate host scripts for both tests (`host_bridge.py` for Test 09 and `host_multimem.py` for Test 10), which is a consequence of the TCB design. The default host is built around a single `Store`, a single linear memory, and a strictly sequential execution model. The composition tests require two sandboxes to be alive simultaneously, each with its own `Store`, each on its own thread, and subject to a different capability regime. First, filesystem permissions rotated between phases for Test 09, and second, a host-mediated channel exposed through explicit function imports for Test 10. Integrating either mechanism into the base `host.py` would conflate the single-guest TCB with the multi-guest TCB and obscure the fact that these are structurally distinct security architectures. We therefore treat each as its own trusted host, with its own interface to the guests it manages.

It is worth noting that we could not have implemented Test 10 by running two instances of `host.py` as separate processes and arranging for them to communicate. Wasmtime's shared-memory primitives are an intra-process concept, which means that they map the same underlying pages into multiple threads of a single host process, but cannot cross an OS process boundary. Two separate Python processes would necessarily fall back to OS-level inter-process communication. The single-process, multi-threaded design of `host_multimem.py` is what allows both sandboxes to share an address space with the host while remaining strictly isolated from one another at the Wasm level.

4.4.1 Test 09: Filesystem Bridge

To validate the filesystem bridge (`08_bridge/transfer_file`), we deploy two separate guest modules that share no direct channel. The Producer opens `/bridge/message.txt` for writing and creates a payload (`LBS_SECRET_TOKEN_2026`). The Consumer opens the same path for reading and echoes the retrieved contents. Neither module possesses any capability beyond the single bridge directory assigned to it for its phase of the workflow.

The evaluation confirms that data is transferred successfully. The Consumer reports the token written by the Producer. The security-relevant observation lies in the asymmetry of authority. The host grants the Producer write access during its execution and then rotates the permission set (via `os.chmod`) to grant the Consumer read-only access before the Consumer is instantiated. This means that when the Consumer holds the bridge capability, it can read the message but is structurally unable to overwrite it, and at no point do both modules simultaneously hold write authority over the shared region. An attempt by either guest to open a path beyond the delegated directory fails.

4.4.2 Test 10: Memory Isolation Between Parallel Sandboxes

To evaluate isolation under concurrent execution (`08_bridge/multi_memory`), we instantiate a Producer and a Consumer on separate threads, each within its own `Store` and private linear memory, and run them in parallel. The two modules communicate exclusively through the host-owned channel via the imported `channel_write` and `channel_read` functions. The Producer computes a result in its private memory and streams it byte-by-byte into the channel, terminating the transfer with a flag. The Consumer polls the channel for the flag and reconstructs the message into its own private buffer.

The evaluation demonstrates two distinct properties. First, the message is transferred correctly while both modules are live, confirming that the channel functions as a usable concurrent communication primitive. Second, and more importantly, the Producer's private state is never exposed: the only bytes the Consumer can observe are those the Producer explicitly wrote to the host-owned buffer. The Consumer holds no memory import aliasing the Producer's address space, and any pointer it constructs is interpreted only relative to its own linear memory. An attempt to address an offset beyond the channel bounds does not silently read adjacent memory; instead the host's bounds check rejects the request. This confirms that even two simultaneously executing adversaries cannot exfiltrate data from one another. The isolation guarantee of section 3.4 therefore holds compositionally, not merely for a module considered in isolation.

5 Related Work

Our work builds on several foundational concepts in language-based security and draws parallels with recent efforts to enforce least-privilege execution in modern software ecosystems. This section surveys the most relevant prior work and positions our contribution within the broader landscape.

Regarding SFI, the concept of isolating untrusted software modules to prevent memory faults from escalating was introduced by [Wahbe et al., 1993]. Their original technique relied on binary rewriting to insert runtime checks that confine memory accesses and control transfers to a designated address range. While effective, this approach required careful instrumentation of native binaries and was tightly coupled to the target architecture. WebAssembly achieves the same isolation guarantees structurally, through its split-memory design and type-safe instruction set, without requiring any binary rewriting. Our runtime leverages this structural SFI to contain classical memory corruption exploits that would succeed on native x86.

To guarantee memory interoperability in WebAssembly, [Fitzgibbons et al., 2024] present RichWasm, a richly typed intermediate language built upon WebAssembly that enables safe, shared-memory interoperability between high-level languages with differing memory management models (e.g., garbage-collected versus manually managed). To prevent cross-boundary memory safety violations, RichWasm incorporates linear capability types at the compilation level that statically track memory ownership, size bounds, and strong updates. While RichWasm focuses on statically enforcing safe interoperability between distinct, cooperating compiled languages sharing a single memory space, our project addresses a distinct adversarial threat model. Rather than statically verifying shared-memory access, we focus on dynamic, execution-level isolation: we treat the compiled Wasm module as an adversarial black box and use host-mediated capabilities and structural SFI to strictly separate the untrusted guest's memory from the TCB.

The motivation for our capability-based approach traces back to the Confused Deputy problem, which [Hardy, 1988] identified as a situation where a privileged program is manipulated by an unprivileged entity into misusing its authority. Hardy's analysis demonstrated that ambient authority, privileges granted implicitly based on the identity of the executing process, is fundamentally incompatible with secure composition of software components. The capability model, in which authority is conveyed through unforgeable tokens rather than ambient identity, directly addresses this vulnerability. Our runtime implements this model through

WASI's capability-based file descriptor system and extends it with ephemeral sandboxing to achieve per-file and per-operation granularity.

A contrasting strategy for securing native binaries comes from [Hybel, 2017], who demonstrated the difficulty of retrofitting security onto native C binaries through x86 assembly rewriting. That approach requires deep knowledge of the target architecture's calling conventions and instruction encoding, and must handle edge cases arising from compiler optimizations and hand-written assembly. Our project pursues a fundamentally different strategy. Rather than securing the native output, we change the compilation target entirely. By compiling C to WebAssembly, we shift the burden of enforcement from fragile binary instrumentation to the structurally sound Wasm virtual machine, where isolation is a property of the execution model rather than an artifact of inserted checks.

Capability-based principles have also been applied at the level of scripting languages. [Moore et al., 2014] present SHILL, a secure shell scripting language that enforces the Principle of Least Privilege through language-level capabilities and contracts. In SHILL, scripts access system resources only through capabilities received from the script invoker, and contracts declaratively specify what capabilities a script requires and how it intends to use them. The SHILL runtime enforces these contracts through a combination of language design and capability-based sandboxing on FreeBSD. Our work shares SHILL's philosophical commitment to zero ambient authority and capability-mediated access, but applies these principles at the WebAssembly level rather than the shell scripting level. Where SHILL relies on OS-level sandboxing primitives (Capsicum on FreeBSD), we leverage WASI's built-in capability model and extend it with ephemeral sandboxing to achieve fine-grained per-file enforcement.

Regarding Supply Chain Security and Runtime Capability Enforcement, [Cornelissen and Balliu, 2025] propose NodeShield, a runtime protection mechanism for Node.js that enforces an application's dependency hierarchy and controls access to system resources at the package level. NodeShield introduces the Capability Bill of Materials (CBOM), an extension of the Software Bill of Materials (SBOM) standard that records the required capabilities of each dependency. At runtime, NodeShield prevents components from accessing system resources beyond their declared capabilities, using a technique called code outlining that requires no modifications to application code or the Node.js runtime. Their evaluation shows that NodeShield can prevent over 98% of 67 known supply chain attacks while incurring minimal overhead. Our work addresses a complementary threat model: where NodeShield enforces capability boundaries between JavaScript packages within a single Node.js process, our runtime enforces capability boundaries between compiled C modules and the host operating system through WebAssembly's structural isolation.

Finally, [Ferreira et al., 2021] propose a lightweight permission system for Node.js that restricts packages' access to security-critical APIs such as the filesystem and network. Their key insight is that many npm packages perform only simple computations and do not require privileged access, creating an opportunity to enforce least-privilege design at the package level with negligible overhead. Their system protects applications by blocking unauthorized API calls from packages that have not been granted the corresponding permission. While their approach operates within the JavaScript runtime and focuses on the npm supply chain, it shares our core design philosophy: stripping ambient authority from untrusted components and requiring explicit capability grants for any privileged operation. Our work applies this same principle at a lower level of abstraction, using WebAssembly and WASI to enforce capability boundaries at the system call interface rather than at the JavaScript API level.

6 Conclusion

This paper presented a secure, capability-based WebAssembly runtime designed to enforce three foundational Language-Based Security properties against adversarial C payloads. By compiling deliberately vulnerable C programs to the `wasm32-wasip1` target and embedding them within a zero-ambient-authority Python host, we demonstrated that each class of attack in our threat model is structurally neutralized.

SFI is enforced by Wasm's split-memory architecture: stack buffer overflows and indirect call hijacking, which would succeed on native x86, are contained without host compromise. The linear memory is entirely decoupled from the VM's internal execution stack, guaranteeing Control Flow Integrity as a structural property rather than an artifact of inserted runtime checks. Execution Monitoring is enforced through Wasmtime's deterministic fuel mechanism, which bounds the computational budget of every guest invocation and guarantees that the host cannot be held hostage by a non-terminating module.

Object Capabilities are enforced by stripping the guest execution environment of all ambient authority and mediating every resource interaction through explicitly delegated capability tokens. We extended WASI P1's coarse-grained directory delegation into a fine-grained, per-file and per-operation access control system using ephemeral sandboxing, where the TCB constructs a minimal, temporary filesystem view for each guest invocation and enforces read, write, or read-write permissions through native OS-level controls. Network access is blocked

both structurally, WASI P1 exposes no socket primitives, and at the capability level, since no system directories are pre-opened.

Finally, we demonstrated that these guarantees compose safely across multiple mutually untrusted sandboxes, both through a sequential filesystem bridge with dynamically rotated permissions and through a concurrent host-mediated channel that preserves SFI by routing every cross-sandbox byte through a host-owned, bounds-checked buffer. In all cases, the host remains the sole reference monitor for every cross-domain operation.

Our evaluation, comprising ten adversarial test cases, confirms that the runtime successfully contains memory corruption, prevents capability escalation, defeats computational denial of service, and enforces strict information flow boundaries between composed sandboxes, all without requiring complex assembly rewriting or native runtime instrumentation.

References

[Cornelissen and Balliu, 2025] Cornelissen, E. and Balliu, M. (2025). NodeShield: Runtime enforcement of security-enhanced SBOMs for Node.js. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*. ACM.

[Ferreira et al., 2021] Ferreira, G., Jia, L., Sunshine, J., and Kästner, C. (2021). Containing malicious package updates in npm with a lightweight permission system. In *Proceedings of the 43rd International Conference on Software Engineering (ICSE)*. IEEE/ACM.

[Fitzgibbons et al., 2024] Fitzgibbons, M., Paraskevopoulou, Z., Mushtak, N., Thalakottur, M., Manzur, J. S., and Ahmed, A. (2024). Richwasm: Bringing safe, fine-grained, shared-memory interoperability down to webassembly. *Proceedings of the ACM on Programming Languages*, 8(PLDI):1–24.

[Hardy, 1988] Hardy, N. (1988). The confused deputy: (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review*, 22(4):36–38.

[Hybel, 2017] Hybel, T. (2017). Securing C programs with assembly rewriting. Technical report, Aarhus University. Language-Based Security Project Report.

[Moore et al., 2014] Moore, S., Dimoulas, C., King, D., and Chong, S. (2014). SHILL: A secure shell scripting language. In *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. USENIX Association.

[Saltzer and Schroeder, 1975] Saltzer, J. H. and Schroeder, M. D. (1975). The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308.

[Wahbe et al., 1993] Wahbe, R., Lucco, S., Anderson, T. E., and Graham, S. L. (1993). Efficient software-based fault isolation. In *Proceedings of the Fourteenth ACM Symposium on Operating Systems Principles*, pages 203–216. ACM.

[Xavier et al., 2026] Xavier, R., Engels, J., and Mureşan, D. (2026). Capability-based WebAssembly runtime implementation. `https://github.com/RuiXavier/LBS2026-Group10-Capability-Wasm`.